AlphaLab Development Roadmap

Phase-by-Phase Implementation Plan

Overview

AlphaLab will be built incrementally in clearly defined phases.

Each phase has a single objective and should be completed before moving to the next.

Every phase follows the same engineering workflow:

1. Plan
2. Implement
3. Test
4. Review
5. Update Documentation
6. Update ADRs (if required)
7. Update Learning Notes
8. Update File References
9. Update Development Log

Only after all of the above are complete is a phase considered finished.

Phase 0 — Engineering Foundation

Objective

Build the engineering foundation for the project.

No business logic.

No APIs.

No factor engine.

No DSL.

Focus entirely on project structure, engineering workflow, documentation, and infrastructure.

Tasks

Repository

- Create repository structure
- Organize packages
- Create documentation structure
- Create infrastructure folder
- Create testing folder

GitHub

- Configure repository
- Create GitHub Project
- Create issue templates
- Create PR template
- Configure labels
- Configure branch protection
- Configure milestones

Git Workflow

- Create development branch
- Define branching strategy
- Define review process
- Define merge policy

Commit Convention

Allowed commit types

- feature(scope):
- bug(scope):
- chore(scope):

Documentation

Create

- README
- CONTRIBUTING
- 00_MASTER_PLAN.md
- 01_ARCHITECTURE.md
- 02_CURRENT_STATE.md
- 03_NEXT_STAGE.md

Create documentation directories

- adr/
- learning_notes/
- file_reference/
- development_log/

ADRs

Write

- ADR-001 Repository Structure
- ADR-002 Documentation Policy
- ADR-003 Project Architecture

Infrastructure

- Docker Compose
- PostgreSQL container
- Redis container
- Optional pgAdmin

CI/CD

Create GitHub Actions skeleton

- Lint
- Tests
- Build

Deliverables

- Complete engineering foundation
- Complete documentation framework
- Development workflow established

Phase 1 — Data Foundation

Objective

Build the complete market data layer.

Everything required for obtaining, validating and storing market data.

Tasks

Market Data Provider

Design provider abstraction

Implement

- MarketDataProvider
- YahooProvider

Universe

Create Universe abstraction.

Implement

- Universe interface
- NIFTY50Universe

Universe must NOT be hardcoded.

Support point-in-time constituents.

Prevent survivorship bias.

Data Validation

Implement

- Missing data detection
- Schema validation
- Data quality checks
- Corporate action detection
- Suspicious price movement detection

DuckDB

Design schema

Create

- OHLCV table
- Universe table
- Factor value table

Ingestion Pipeline

Build

Yahoo

↓

Validation

↓

Transformation

↓

DuckDB

Documentation

Learning Notes

- Market Data
- Provider Pattern
- Universe
- Corporate Actions
- DuckDB

ADRs

- Market Data Provider
- DuckDB
- Universe

Deliverables

Complete data ingestion pipeline.

Phase 2 — Factor DSL

Objective

Design a secure DSL for defining alpha factors.

Tasks

Language Design

Define grammar

Supported primitives

- Momentum
- Volatility
- Rolling Mean
- Rolling Std

Arithmetic

- +
- -
- *
- /

Compiler

Implement

Lexer

↓

Parser

↓

AST

↓

Validator

↓

Compiler

↓

Executable Function

Validator

Detect

- Syntax errors
- Look-ahead bias
- Invalid windows
- Complexity violations

Testing

Unit tests

Parser tests

Compiler tests

Validation tests

Documentation

Learning Notes

- DSL
- Compiler Design
- AST
- Parsing
- Lexing
- Validation

ADR

Factor DSL

Deliverables

Fully working DSL compiler.

Phase 3 — Backtesting Engine

Objective

Implement the research evaluation engine.

Tasks

Implement

- Walk-forward validation
- Rolling windows
- Expanding windows

Metrics

- Sharpe
- Sortino
- Calmar
- Max Drawdown
- IC
- Rank IC
- Turnover

Generate

- Equity curve
- Performance summary

Documentation

Learning Notes

- Alpha Factors
- Walk-forward Validation
- Information Coefficient
- Sharpe Ratio
- Backtesting

ADR

Backtesting Engine

Deliverables

Complete backtesting engine.

Phase 4 — Background Execution

Objective

Move long-running computation into asynchronous jobs.

Tasks

Configure

- Celery
- Redis

Implement

- Job Queue
- Job Status
- Progress Tracking
- Failure Handling

Integrate

Backtesting

↓

Celery

↓

Results

Documentation

Learning Notes

- Celery
- Redis
- Async Processing

ADR

Task Queue

Deliverables

Background execution pipeline.

Phase 5 — Robustness Engine

Objective

Implement the core research thesis.

Tasks

Stress Tests

Noise Injection

- 0.5%
- 1%
- 2%

Missing Data

- 5%
- 10%
- 20%

Generate

- Robustness Score
- Failure Reasons
- Stability Analysis

Documentation

Learning Notes

- Robustness
- Noise Injection
- Missing Data
- Research Evaluation

ADR

Robustness Engine

Deliverables

Complete robustness evaluation.

Phase 6 — Backend API

Objective

Connect every subsystem through FastAPI.

Tasks

Authentication

Users

Experiments

Factors

Backtests

Robustness

Leaderboard

Research Reports

Status APIs

Validation

Database Integration

Error Handling

Documentation

Learning Notes

- FastAPI
- REST APIs
- Dependency Injection
- Request Lifecycle

ADR

Backend Architecture

Deliverables

Complete backend.

Phase 7 — Research Reports

Objective

Automatically generate research artifacts.

Tasks

Generate

Factor Summary

Performance Metrics

Stress Results

Robustness Score

Failure Explanation

Recommendations

Support

Markdown

PDF

Documentation

Learning Notes

Research Reporting

ADR

Research Report Generator

Deliverables

Automatic report generation.

Phase 8 — Frontend

Objective

Build the user interface.

Tasks

Authentication

Dashboard

Leaderboard

Factor Detail

Research Report Viewer

Charts

Heatmaps

Loading States

Error Handling

Responsive Design

Documentation

Learning Notes

Next.js

Frontend Architecture

State Management

Charts

ADR

Frontend Architecture

Deliverables

Fully functional frontend.

Phase 9 — Testing

Objective

Ensure correctness of the entire system.

Tasks

Unit Tests

Integration Tests

API Tests

Database Tests

DSL Tests

Research Validation Tests

Performance Tests

Coverage Reports

Documentation

Testing Strategy

Test Architecture

ADR

Testing Strategy

Deliverables

Reliable, tested system.

Phase 10 — Deployment

Objective

Deploy AlphaLab.

Tasks

GitHub Actions

Render

Vercel

Neon

Environment Variables

Docker

Production Configuration

Monitoring Preparation

Release Pipeline

Documentation

Deployment Guide

Operations Guide

ADR

Deployment Architecture

Deliverables

Production deployment.

Phase 11 — Polish & Interview Readiness

Objective

Prepare AlphaLab as a flagship portfolio project.

Tasks

Improve Documentation

Review ADRs

Review Learning Notes

Review File References

Review Development Logs

Improve UI

Improve Performance

Refactor

Code Cleanup

Repository Audit

Interview Defense Review

Resume Updates

Demo Preparation

Architecture Review

Final Deliverables

- Complete documentation
- Complete engineering notes
- Complete file references
- Complete ADRs
- Complete development logs
- Production-ready repository

- Interview-ready architecture
- Portfolio-ready project

Definition of Done (Applies to Every Phase)

A phase is complete only when all of the following have been completed:

- Implementation completed
- Unit tests written
- Integration tests updated (if applicable)
- Code reviewed
- Architecture documentation updated
- Current state updated
- Next stage rewritten
- ADRs updated (if required)
- Learning Notes written
- File Reference documentation written
- Development Log updated
- CI passes successfully
- Repository remains buildable
- Documentation and implementation are synchronized